
Artefato Sprint 2 — XSS (Cross-Site Scripting)

OficinaTech — Sistema de Ordens de Serviço

Grupo: [preencher]

Integrantes:

[Nome completo — Matrícula]

[Nome completo — Matrícula]

[Nome completo — Matrícula]

[Nome completo — Matrícula]

Repositório do projeto (GitHub):

<https://github.com/eduHanjos0411/os-xss-lab>

Brasília — 2026

Conformidade com a Rubrica de Avaliação

A tabela abaixo mapeia cada critério da rubrica do guia (Seção 4) ao item do projeto que o atende. A dimensão **D — Apresentação (vídeo)** não é coberta por este relatório, por não fazer parte do escopo documental.

Critério	Pts	Onde é atendido no projeto	OK
A. XSS tipo 1 — Reflected	10	Busca refletida via URL ?busca=... (codigo-vulneravel/app.js:136-141)	Sim
A. XSS tipo 2 — Stored	10	Campos da OS renderizados com innerHTML no card e no modal (app.js:86, 112)	Sim
A. Banco de dados adequado	5	Persistência em localStorage (os_vuln) + cookie de sessão exfiltrável	Sim
A. Aplicação funcional	5	Fluxo normal (cadastro, busca, detalhes, paginação) operante	Sim
B. Ataque 1 documentado	10	Seção 3.1 (Reflected): payload, contexto, resultado	Sim
B. Ataque 2 documentado	10	Seção 3.2 (Stored): payload, contexto, resultado	Sim
B. Análise STRIDE	5	Seções 3 e 4 (tabela STRIDE consolidada)	Sim
B. Clareza e completude	5	Todas as seções do template preenchidas	Sim
C. Correção ponto 1 (escaping/sanit.)	8	textContent + createElement nos cards e na busca (codigo-corrigido/app.js)	Sim
C. Correção ponto 2 (método diferente)	8	Output encoding com escaparHTML() no modal (app.js:43-50, 146-150)	Sim
C. Verificação da correção	5	Seção 5.4 (mesmos payloads viram texto literal)	Sim
C. Defesas adicionais	4	CSP em codigo-corrigido/index.html:23-24 (script-src 'self')	Sim

Itens de bônus atendidos

- **Roubo de cookie com Reflected XSS (+3):** o payload de busca lê e exibe document.cookie (cookie sessao criado pela app).
- **Content Security Policy como defesa (+4):** CSP declarada no cabeçalho da versão corrigida.

Observação honesta: não implementamos um terceiro tipo (DOM-based) como ponto independente, então o bônus de +3 por terceiro tipo não é reivindicado.

1. Descrição da Aplicação

Nome e propósito. OficinaTech é um sistema de acompanhamento de Ordens de Serviço (OS) de manutenção de eletrônicos. Cada OS contém cliente, equipamento, data, defeito e descrição. Há um campo de pesquisa no topo, os cards das ordens no centro e um modal de **Detalhes** por OS.

Tecnologias. HTML5, CSS3 e JavaScript puro (Vanilla JS), sem framework e sem backend. A stack corresponde à opção recomendada pelo guia (HTML + CSS + JS puro).

Estrutura de dados. Como não há servidor, a persistência usa o armazenamento do navegador, que faz o papel de 'banco de dados' da aplicação:

- `localStorage['os_vuln']` — lista de OS da versão vulnerável (JSON).
- `localStorage['os_seguro']` — lista da versão corrigida (chave distinta: dados desacoplados).
- `cookie 'sessao'` — token de sessão fictício criado pela versão vulnerável, usado para demonstrar o roubo de cookie.

Duas versões, mesma origem. O projeto tem `codigo-vulneravel/` e `codigo-corrigido/`, servidas sob a mesma origem para uma comparação honesta.

Como executar localmente

Suba um servidor estático na **raiz** do projeto (não dentro de uma subpasta):

```
# Opção A — Python
python -m http.server 8000
```

```
# Opção B — VS Code Live Server (abrir a pasta raiz os-xss-lab)
```

Depois acesse:

- Vulnerável: `http://localhost:8000/codigo-vulneravel/`
- Segura: `http://localhost:8000/codigo-corrigido/`

Abrir por file:// não serve: a CSP e o conceito de origem não funcionam igual. Mantenha o mesmo host (não misturar localhost e 127.0.0.1), pois cada origem tem seu próprio localStorage.

2. Pontos de XSS Identificados

#	Localização	Parâmetro / Campo	Tipo	Contexto
1	Barra de busca (refletida na URL)	GET busca	Reflected	Conteúdo HTML (innerHTML)
2	Campos da OS: card e modal de detalhes	cliente, equipamento, defeito, descrição	Stored	Conteúdo HTML (innerHTML)

Ponto 1 — Reflected. Ao pesquisar, a aplicação grava o termo na query string (`?busca=...`) e recarrega. No carregamento, o termo é lido da URL e refletido na página via `innerHTML`. O payload vive no link — basta a vítima abrir a URL para o script executar.

Ponto 2 — Stored. Os campos da OS são salvos crus no `localStorage` e reinjetados via `innerHTML` em dois sinks: ao renderizar o card e ao abrir o modal de Detalhes. O payload fica persistido e dispara toda vez que a OS é exibida.

3. Ataques Documentados

3.1 Ataque 1 — Reflected XSS (busca via URL)

Objetivo: executar script e vaziar o cookie de sessão de quem abrir um link malicioso.

Tipo: Reflected XSS. **Contexto:** injeção em conteúdo HTML via innerHTML.

Payload (digitado na busca ou embutido no link):

```
&lt;img src=x onerror="alert(document.cookie)"&gt;
```

Código vulnerável (codigo-vulneravel/app.js:136-141):

```
document.getElementById('resultado-busca').innerHTML =  
  'Exibindo resultados para: &lt;strong&gt;' + termo + '&lt;/strong&gt;';
```

Resultado: a página recarrega com ?busca=<img...> na URL; o onerror dispara e o alert exibe o cookie sessao=abc123-token-fake. Copiando a URL e abrindo em outra aba, o ataque dispara de novo — provando que o vetor é o link.

STRIDE: S (roubo de sessão via cookie), I (vazamento do cookie), T (alteração do DOM da página).

3.2 Ataque 2 — Stored XSS (campos da OS)

Objetivo: persistir um script que executa para qualquer um que visualize a OS.

Tipo: Stored XSS. **Contexto:** injeção em conteúdo HTML via innerHTML (dois sinks: card e modal).

Payload (no campo **Defeito** e/ou **Descrição** do formulário de nova OS):

```
&lt;img src=x onerror="alert('XSS armazenado')"&gt;
```

Código vulnerável (card — codigo-vulneravel/app.js:86; modal — app.js:112):

```
card.innerHTML = `  
  &lt;h3&gt;${os.cliente}&lt;/h3&gt;  
  &lt;span class="linha"&gt;[AVISO] ${os.defeito}&lt;/span&gt;  
  ...`;
```

Resultado: no campo **Defeito**, o alert dispara assim que o card é renderizado; ao pressionar F5 o alert volta, provando a persistência no localStorage. No campo **Descrição**, o alert dispara ao clicar em **Detalhes** (segundo sink).

STRIDE: S, I, T; R parcialmente (ações em nome do usuário).

Nota de honestidade: como a app é client-side, o 'stored' persiste no navegador de quem gravou — é uma simulação fiel do mecanismo de Stored XSS, não um vazamento multiusuário real. A vulnerabilidade em si é real: o innerHTML executa o payload.

4. Análise de Impacto

Tabela STRIDE consolidada

Categoria	Aplica?	Justificativa no contexto da app
S — Spoofing	Sim	Com document.cookie o atacante captura a sessão e se passa pela vítima.
T — Tampering	Sim	O script altera o conteúdo/DOM exibido ao usuário.
I — Information Disclosure	Sim	Vaza cookie, localStorage e dados em tela.
R — Repudiation	Parcial	Pode executar ações em nome do usuário sem seu conhecimento.
D — Denial of Service	Não	Não é o foco (sem loops destrutivos demonstrados).
E — Elevation of Privilege	Não	App não possui área administrativa/privilegios.

Impacto nos dados e no negócio

- **Dados comprometidos:** cookie de sessão, conteúdo do localStorage (todas as OS) e qualquer dado visível na tela.
- **Impacto no negócio:** sequestro de sessão de atendentes, adulteração de ordens de serviço, perda de confiança e exposição de dados de clientes.
- **Contexto LGPD (Lei 13.709/2018):** nomes de clientes e dados de equipamentos são dados pessoais; seu vazamento por XSS poderia configurar incidente de segurança com dever de notificação à ANPD e risco de sanções.

5. Correções Implementadas

A versão corrigida mantém as mesmas funcionalidades e aplica **métodos de defesa diferentes** em cada ponto, além de uma camada extra de CSP.

5.1 Ponto 1 — Cards e busca: textContent / createElement

Em vez de concatenar HTML, cada nó é criado e preenchido com textContent, que trata o conteúdo como texto literal.

Vulnerável:

```
card.innerHTML = `<h3>${os.cliente}</h3> ...`;
```

Corrigido (codigo-corrigido/app.js:90-96):

```
const h3 = document.createElement('h3');
h3.textContent = os.cliente; // texto literal => seguro
card.appendChild(h3);
```

A busca refletida também passou a usar textContent (app.js:169-172), então o termo da URL aparece escrito na tela, nunca executa.

5.2 Ponto 2 — Modal: output encoding com escaparHTML() (método diferente)

Onde ainda é conveniente montar markup como string, todo valor do usuário passa por escaparHTML(), que converte < > & " ' em entidades — equivalente ao htmlspecialchars() do PHP.

Corrigido (codigo-corrigido/app.js:43-50 e 146-150):

```
function escaparHTML(valor) {
  return String(valor)
    .replace(/&/g, '&amp;').replace(/</g, '&lt;')
    .replace(/>/g, '&gt;').replace(/"/g, '&quot;')
    .replace(/'/g, '&#39;');
}
// uso: <h3>OS #${escaparHTML(os.id)} - ${escaparHTML(os.cliente)}</h3>
```

5.3 Defesa adicional — Content Security Policy

A versão corrigida declara uma CSP no cabeçalho (codigo-corrigido/index.html:23-24):

```
<meta http-equiv="Content-Security-Policy"
  content="default-src 'self'; script-src 'self'; object-src 'none'; base-uri 'none'">
```

Mesmo que algum payload escapasse, script-src 'self' bloquearia scripts inline e de origens externas — defesa em profundidade.

5.4 Verificação da correção

Repetindo os **mesmos payloads** dos ataques na versão segura:

- Busca e cards: o texto `<img src=x onerror=...>` aparece **literal** na tela, sem alert.
- Modal: idem, graças ao escape.
- A regra geral: se o navegador **mostra o script como texto em vez de executá-lo**, a correção está funcionando — exatamente o critério do guia.

6. Conclusão

O artefato demonstrou, na mesma aplicação, dois tipos de XSS (Reflected via URL e Stored nos campos da OS) e suas correções. A causa raiz é única: dado controlado pelo usuário inserido na página via `innerHTML`, fazendo o navegador interpretá-lo como código. A lição central é que **o contexto define a defesa**: usar `textContent` para texto, `output encoding` para quando é preciso montar markup, e CSP como reforço. A comparação lado a lado entre as pastas `codigo-vulneravel/` e `codigo-corrigido/` evidencia que a mesma funcionalidade pode ser entregue de forma segura.

Código-fonte completo: <https://github.com/eduHanjos0411/os-xss-lab>